SWASTYASATHI

Health Analysis & AI-Assisted Telehealth System Design

Technical Architecture, Data Model, Analysis Pipeline, AI Integration, Offline-First Design and Implementation Roadmap

Project: SwastyaSathi — Offline-First Telemedicine Platform for Rural Nepal

Initial health scope: Cardiovascular/metabolic monitoring with a focus on blood pressure and diabetes-related data.

Application stack: Flutter + Node.js/TypeScript + PostgreSQL + Python/FastAPI + LLM API

Document purpose

This document translates the proposed health-analysis architecture into an implementation-oriented system design. It is intended to guide database evolution, backend/API development, Python analytics, LLM integration, testing, and future clinical review.

Important clinical-design principle

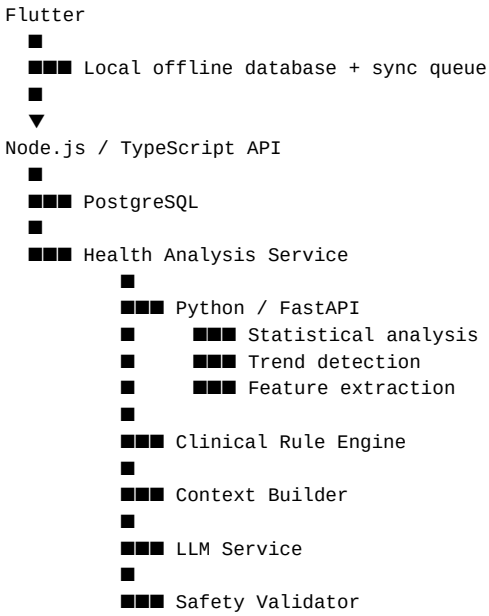
> The system should not be designed as an autonomous AI doctor. Deterministic calculations and clinician-reviewed rules should produce the health-analysis facts; the LLM should primarily explain, summarize, personalize and communicate those validated results.

1. Executive Summary

SwastyaSathi already has a strong core telemedicine schema covering users, patients, doctors, appointments, consultations, prescriptions, pharmacy, payments, notifications and audit logs. The main gap is the longitudinal health-data layer required for meaningful analysis.

The recommended design separates raw health measurements from clinical analysis and from AI-generated explanations. Blood pressure, glucose, weight, symptoms, conditions and medications are stored as time-aware records. A Python analytics service calculates statistics and trends, a clinical rule engine produces structured flags, and an LLM converts that structured context into user-friendly feedback. A safety validator sits between AI output and the mobile client.

The target architecture is:



> The architecture intentionally avoids using the LLM for arithmetic, raw-data aggregation, or unsupervised clinical decision-making.

2. Goals and Non-Goals

2.1 Goals

- Capture longitudinal blood-pressure, glucose and weight measurements with timestamps and measurement context.
- Maintain current medication information separately from historical prescriptions.
- Support symptom and condition context that can be correlated with measurements.
- Calculate transparent statistics and trends before invoking an LLM.
- Generate structured health-analysis records that can be reproduced and versioned.
- Provide understandable AI-assisted explanations without making the LLM the source of truth.
- Support offline-first entry and reliable synchronization.
- Maintain auditability, security and data lineage appropriate for a telehealth system.

2.2 Non-Goals for the first version

- Autonomous diagnosis.
- Autonomous medication prescribing, stopping or changing.
- Using an LLM as the numerical/statistical analysis engine.
- Predictive clinical machine learning before sufficient validated data and governance exist.
- Replacing clinician judgment.

3. Existing Database Assessment

The supplied PostgreSQL schema is a good platform foundation. The following areas can remain substantially unchanged:

Area	Assessment	Action
Identity/access	Strong foundation	Keep
Patient/doctor profiles	Good foundation	Extend patient health profile
Appointments/consultations	Appropriate	Keep
Prescriptions/pharmacy	Good transactional model	Keep; add active medication layer
Payments/refunds	Good platform layer	Keep
Notifications/audit	Useful	Keep
Vitals	Too generic for longitudinal analysis	Replace/retire in favor of specialized measurements
Health analysis	Missing	Add
AI feedback/conversations	Missing	Add if AI interaction is part of MVP
Offline synchronization	Not explicitly modeled	Design sync/idempotency strategy

> The most important conceptual change is from patient → vitals to patient → time-series health measurements → analysis → AI explanation.

4. Health Data Domain Model

The health subsystem should treat measurements as events in time. A measurement is not simply the patient's current state; it is an observation made at a specific time, under a specific context and optionally from a specific device.

```
Patient
 ■
 ■■■ Patient Conditions
 ■       ■■■ Type 2 diabetes
 ■       ■■■ Hypertension
 ■
 ■■■ Blood Pressure Readings
 ■       ■■■ systolic
 ■       ■■■ diastolic
 ■       ■■■ heart rate
 ■       ■■■ measurement context
 ■
 ■■■ Glucose Readings
 ■       ■■■ value
 ■       ■■■ unit
 ■       ■■■ measurement type
 ■
 ■■■ Weight Readings
 ■       ■■■ weight_kg
 ■
 ■■■ Symptoms
 ■
 ■■■ Patient Medications
 ■       ■■■ Medication Events / adherence
 ■
 ■■■ Medical Records
 ■
 ■■■ Lab Reports
```

4.1 Static vs dynamic data

Data category	Examples	Storage pattern
Relatively stable profile	Height, DOB, gender, baseline profile	Patient/profile fields
Conditions	Diabetes type, hypertension	Patient-condition records
Frequent measurements	BP, glucose	Time-series tables
Periodic measurements	Weight	Time-series table
Treatment	Current medication	Patient medication
Treatment events	Taken/missed medication	Medication events
Symptoms	Headache, dizziness	Symptom records
AI/analysis	Trends, flags, summaries	Versioned JSONB analysis records

5. Recommended Database Changes

5.1 Blood pressure readings

```
CREATE TABLE blood_pressure_readings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    systolic SMALLINT NOT NULL,
    diastolic SMALLINT NOT NULL,
    heart_rate SMALLINT,
    measured_at TIMESTAMPTZ NOT NULL,
    measurement_context VARCHAR(30),
```

```
    source VARCHAR(30) NOT NULL DEFAULT 'manual',
    device_id UUID,
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (systolic > 0),
    CHECK (diastolic > 0),
    CHECK (heart_rate IS NULL OR heart_rate > 0)
);

CREATE INDEX idx_bp_patient_date
ON blood_pressure_readings(patient_id, measured_at DESC);
```

5.2 Glucose readings

```
CREATE TABLE glucose_readings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    glucose_value NUMERIC(6,2) NOT NULL,
    unit VARCHAR(10) NOT NULL DEFAULT 'mg/dL',
    measurement_type VARCHAR(30) NOT NULL,
    measured_at TIMESTAMPTZ NOT NULL,
    source VARCHAR(30) NOT NULL DEFAULT 'manual',
    device_id UUID,
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_glucose_patient_date
ON glucose_readings(patient_id, measured_at DESC);
```

Recommended measurement types include fasting, before_meal, after_meal, random and bedtime. The exact allowed values should be validated by the application and, once requirements stabilize, by an enum/reference table.

5.3 Weight readings

```
CREATE TABLE weight_readings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    weight_kg NUMERIC(6,2) NOT NULL,
    measured_at TIMESTAMPTZ NOT NULL,
    source VARCHAR(30) NOT NULL DEFAULT 'manual',
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_weight_patient_date
ON weight_readings(patient_id, measured_at DESC);
```

5.4 Patient conditions

```
CREATE TABLE patient_conditions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    condition_name VARCHAR(150) NOT NULL,
    condition_type VARCHAR(50),
    diagnosis_date DATE,
    status VARCHAR(30) NOT NULL DEFAULT 'active',
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

For a longer-term implementation, condition names should be normalized into a reference table and mapped to standard clinical codes where appropriate.

5.5 Active patient medications

```
CREATE TABLE patient_medications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    prescription_item_id UUID REFERENCES prescription_items(id) ON DELETE SET NULL,
    medicine_name VARCHAR(200) NOT NULL,
    dosage VARCHAR(100),
    route prescription_item_route DEFAULT 'oral',
    frequency VARCHAR(100),
    start_date DATE NOT NULL,
    end_date DATE,
    is_active BOOLEAN NOT NULL DEFAULT true,
    instructions TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

This table complements, rather than replaces, the existing prescription model. A prescription answers what a doctor prescribed at a point in time; patient_medications answers what the patient is currently taking or has taken historically.

5.6 Medication adherence events

```
CREATE TABLE medication_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    medication_id UUID NOT NULL REFERENCES patient_medications(id) ON DELETE CASCADE,
    scheduled_at TIMESTAMPTZ NOT NULL,
    taken_at TIMESTAMPTZ,
    status VARCHAR(20) NOT NULL,
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

5.7 Symptoms

```
CREATE TABLE symptom_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    symptom_name VARCHAR(150) NOT NULL,
    severity SMALLINT,
    started_at TIMESTAMPTZ,
    ended_at TIMESTAMPTZ,
    notes TEXT,
    recorded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (severity IS NULL OR severity BETWEEN 1 AND 10)
);
```

5.8 Health devices

The existing devices table should continue to represent app/authentication devices. Do not overload it with medical devices such as blood-pressure monitors.

```
CREATE TABLE health_devices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    device_type VARCHAR(50) NOT NULL,
    manufacturer VARCHAR(100),
    model VARCHAR(100),
    serial_number VARCHAR(100),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

6. Analysis Data Model

The output of the deterministic analysis engine should be persisted independently from raw measurements. This allows the application to show previous analyses, reproduce results, and track which algorithm/rule version produced them.

```
CREATE TABLE health_analyses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    analysis_type VARCHAR(50) NOT NULL,
    period_start TIMESTAMPTZ NOT NULL,
    period_end TIMESTAMPTZ NOT NULL,
    engine_version VARCHAR(50) NOT NULL,
    rule_version VARCHAR(50),
    input_snapshot JSONB,
    result JSONB NOT NULL,
    generated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (period_end >= period_start)
);
```

The input_snapshot should contain the compact inputs used by the analysis, not an uncontrolled dump of the entire patient database. The result should contain structured facts such as averages, trends, counts and rule flags.

7. AI / LLM Data Model

The LLM should not become the source of truth for clinical information. It receives a controlled health context generated by the backend and analysis engine.

```
CREATE TABLE ai_health_feedback (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    analysis_id UUID REFERENCES health_analyses(id) ON DELETE SET NULL,
    model_provider VARCHAR(50),
    model_name VARCHAR(100),
    prompt_version VARCHAR(50),
    context JSONB,
    response JSONB,
    generated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

If the application includes a conversational AI feature, use separate conversation and message tables:

```
CREATE TABLE ai_conversations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    title VARCHAR(200),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

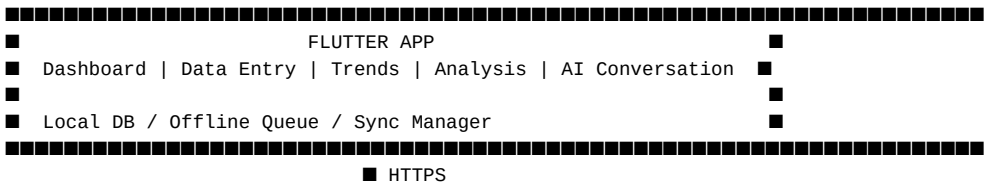
CREATE TABLE ai_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id UUID NOT NULL REFERENCES ai_conversations(id) ON DELETE CASCADE,
    role VARCHAR(20) NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

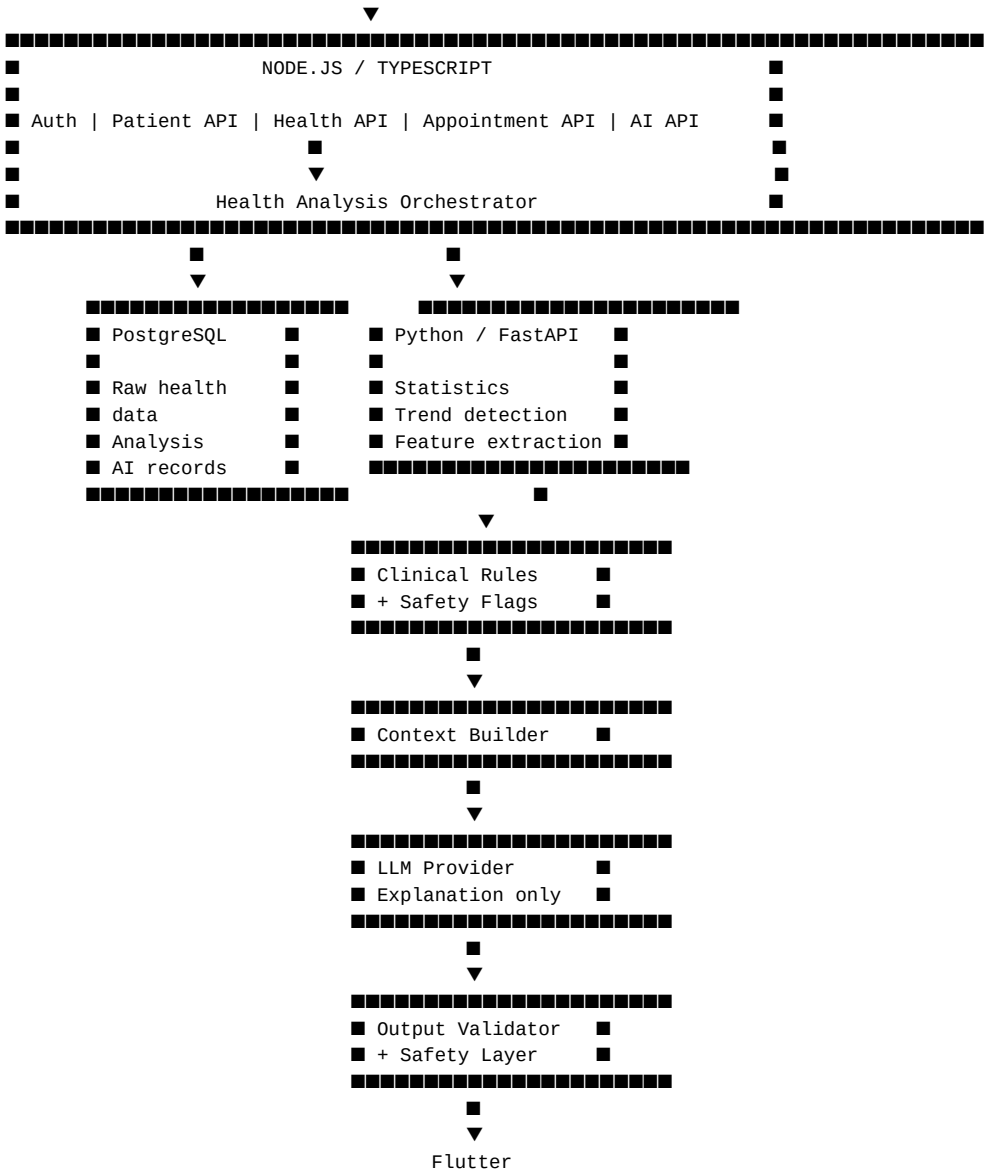
8. Source-of-Truth Model

Information	Authoritative source
Raw BP/glucose/weight measurements	Measurement tables
Current medication list	patient_medications
Historical prescriptions	prescriptions + prescription_items
Conditions	patient_conditions / clinician records
Computed trends	health_analyses
AI explanation	ai_health_feedback
Conversation history	ai_conversations + ai_messages
Clinical rules	Version-controlled rule definitions/code reviewed by qualified clinicians

> Rule: Never treat an LLM response as a replacement for a structured clinical fact already stored in the database.

9. End-to-End System Architecture




NODE.JS / TYPESCRIPT
Auth | Patient API | Health API | Appointment API | AI API
Health Analysis Orchestrator
PostgreSQL
Raw health data
Analysis
AI records
Python / FastAPI
Statistics
Trend detection
Feature extraction
Clinical Rules
+ Safety Flags
Context Builder
LLM Provider
Explanation only
Output Validator
+ Safety Layer
Flutter

10. Health Analysis Pipeline

The core principle is: raw data → deterministic analysis → clinical rules → structured context → LLM explanation → validation.

10.1 Data validation

- Validate required fields and units.
- Reject impossible or malformed values at the API/database boundary.
- Preserve the original measurement timestamp.
- Distinguish measured_at from created_at/upload time.
- Detect duplicate synchronization attempts using client-generated identifiers or idempotency keys.
- Mark source as manual, device, imported or clinician-entered where applicable.

10.2 Statistical analysis

- Latest measurement.
- Measurement count.
- Mean and median where appropriate.
- 7-day, 30-day and 90-day rolling summaries.
- Percent change for weight and other suitable variables.
- Distribution and variability where meaningful.
- Time-series slope/trend estimates when enough observations exist.

10.3 Trend detection

A trend should not be inferred from a single reading. Require a minimum number of observations and a defined time window. A simple initial implementation can use moving averages and linear regression/slope, followed by clinician-reviewed thresholds for what constitutes a meaningful trend.

```
Example conceptual feature output:

{
  "blood_pressure": {
    "measurement_count": 28,
    "latest": {"systolic": 138, "diastolic": 88},
    "average_7d": {"systolic": 134, "diastolic": 85},
    "average_30d": {"systolic": 134, "diastolic": 84},
    "trend": "increasing"
  }
}
```

10.4 Clinical rules

Clinical thresholds and safety rules should be maintained centrally, versioned, and reviewed by qualified clinicians. Avoid scattering threshold values throughout application code.

```
ClinicalRules
  ■■■ BP rules
  ■■■ Glucose rules
  ■■■ Symptom rules
  ■■■ Medication-related rules
  ■■■ Emergency/safety rules

Each rule should have:
  rule_id
```

```
version
effective_date
source/reference
conditions
action/flag
```

> The exact clinical thresholds should be supplied by the project's qualified clinical reviewers and the applicable current guidelines; they should not be invented inside the LLM prompt.

11. LLM Responsibilities

The LLM is best suited to language and interaction tasks:

- Explain structured analysis in plain language.
- Summarize changes over time.
- Answer questions about the displayed analysis.
- Personalize wording using relevant, authorized context.
- Generate questions the patient may want to discuss with a healthcare professional.
- Translate or simplify explanations for supported languages.

The LLM should not be responsible for:

- Calculating averages or trends.
- Inventing thresholds.
- Making medication changes.
- Declaring a diagnosis solely from a prompt.
- Overriding deterministic safety rules.
- Treating conversational memory as authoritative medical history.

12. Health Context Builder

Before each analysis explanation, the backend should construct a compact, task-specific context object.

```
{
  "patient": {
    "age": 45,
    "sex": "...",
    "conditions": ["type_2_diabetes"]
  },

  "medications": [
    {
      "name": "...",
      "dose": "...",
      "frequency": "..."
    }
  ],

  "blood_pressure": {
    "latest": {"systolic": 138, "diastolic": 88},
    "7d": {...},
    "30d": {...},
    "trend": "increasing",
    "measurement_count": 28
  },

  "glucose": {
    "fasting_average_30d": 128,
    "trend": "stable"
  },

  "weight": {
    "current": 71.2,
    "change_30d": -1.3
  },

  "flags": []
}
```

Only retrieve information relevant to the current task. For a question about BP, do not automatically send years of unrelated glucose or conversation history.

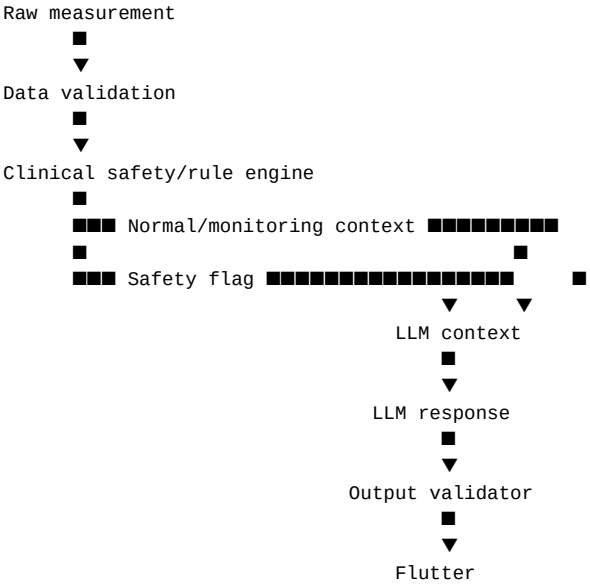
13. LLM Structured Output Contract

Require the model to return a machine-readable object rather than arbitrary text.

```
{
  "summary": "Plain-language summary.",
  "observations": [
    {
      "category": "blood_pressure",
      "message": "...",
      "severity": "info"
    }
  ],
  "recommendations": [
    {
      "type": "monitoring",
      "message": "..."
    }
  ],
  "questions_for_clinician": [
    "..."
  ],
  "safety_message": null
}
```

The Node.js service should validate this schema before returning it to Flutter. If validation fails, use a safe fallback response rather than rendering arbitrary model output.

14. Safety Architecture

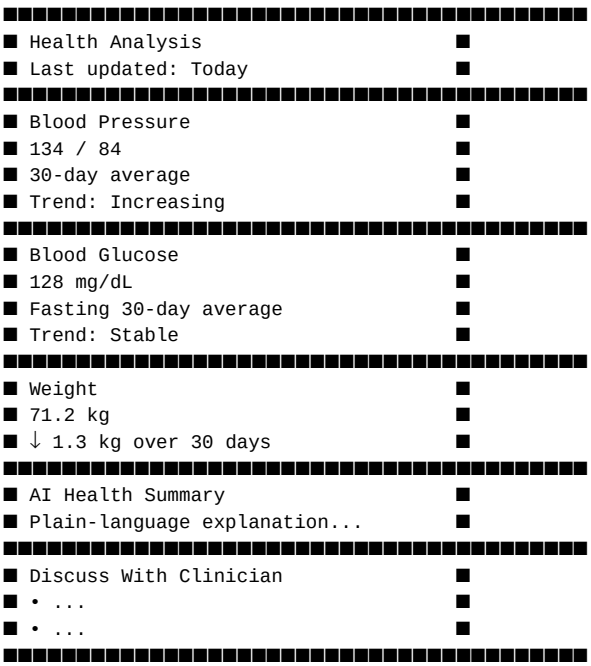


> Critical safety behavior should not depend on the LLM. If a configured rule identifies an urgent situation, the application should use a clinician-approved safety message/template and appropriate escalation behavior.

The system should also prevent the model from changing medication, issuing a diagnosis, or contradicting a higher-priority safety rule.

15. Flutter Analysis Screen

The analysis page should separate objective data from AI interpretation.

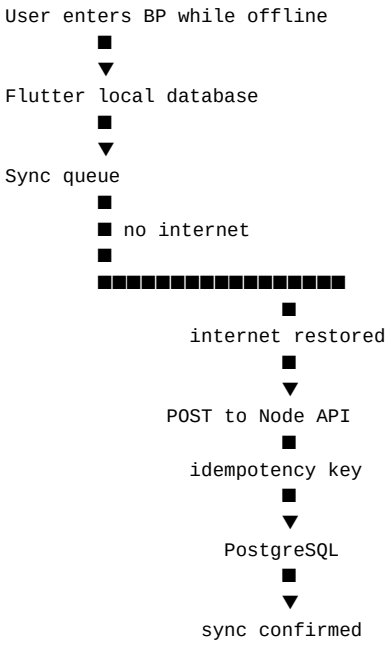


Graphs should show measurement history and, where appropriate, rolling averages. The user should be able to change the date range and inspect the underlying readings.

16. Offline-First Architecture

Because the platform targets rural Nepal, network interruptions should be treated as normal rather than exceptional.

Recommended flow:



For health measurements, the client should generate the record UUID locally. The server should accept that identifier and treat retries as idempotent. This prevents duplicate records when a mobile request succeeds server-side but the acknowledgement is lost.

> Health measurements must preserve measured_at independently from created_at. Example: a BP reading taken at 08:00 and uploaded at 18:00 must remain an 08:00 measurement.

17. API Design

Recommended initial API organization:

```
/api/v1/auth
/api/v1/patients
/api/v1/health/blood-pressure
/api/v1/health/glucose
/api/v1/health/weight
/api/v1/health/symptoms
/api/v1/health/medications
/api/v1/health-analysis/summary
/api/v1/health-analysis/blood-pressure
/api/v1/health-analysis/glucose
/api/v1/ai/conversations
/api/v1/ai/messages

/api/v1/appointments
/api/v1/consultations
/api/v1/prescriptions
/api/v1/pharmacy
/api/v1/payments
```

Example: create a BP measurement

```
POST /api/v1/health/blood-pressure
Authorization: Bearer <token>
Idempotency-Key: <client-record-uuid>
```

```
{
  "systolic": 138,
  "diastolic": 88,
  "heart_rate": 76,
  "measured_at": "2026-09-30T07:30:00+05:45",
  "measurement_context": "morning",
  "source": "manual"
}
```

Example: request an analysis

```
POST /api/v1/health-analysis/summary

{
  "period": "30d"
}
```

> The server should authenticate the patient and derive patient_id from the authenticated identity rather than trusting a patient_id supplied by the client.

18. Python Analytics Service

Python is appropriate for the analytical component because of its mature ecosystem for numerical and statistical processing.

Technology	Purpose
FastAPI	Internal analytics HTTP service
Pandas	Time-series manipulation and aggregation
NumPy	Numerical operations
SciPy	Statistical functions where needed
scikit-learn	Optional future ML/trend models

Recommended internal endpoints:

```
POST /internal/analysis/blood-pressure
POST /internal/analysis/glucose
POST /internal/analysis/weight
POST /internal/analysis/complete
```

The Python service should preferably receive a controlled dataset or authorized internal query result rather than expose the entire PostgreSQL database directly to arbitrary callers.

19. Security and Privacy

- Use HTTPS/TLS for all mobile-to-server and service-to-service traffic.
- Hash passwords with a modern password hashing algorithm; never store plaintext passwords.
- Keep LLM/API credentials in server-side secret management, never in Flutter.
- Send only the minimum necessary information to an external LLM.
- Do not send patient names, addresses, phone numbers or other direct identifiers unless genuinely required.
- Use role-based authorization for patient, doctor and admin actions.
- Maintain audit logs for sensitive operations.
- Restrict internal Python and LLM endpoints so they are not publicly callable without authorization.
- Encrypt sensitive stored data and backups according to the deployment environment.
- Define data retention, deletion and access policies before production deployment.

> Because this system handles health information, privacy and security requirements should be reviewed against the laws, regulations, contractual requirements and clinical governance applicable to the deployment jurisdiction.

20. Observability and Auditability

A health-analysis system should make it possible to answer: what data was used, which algorithm version was used, which rules were active, which model generated the explanation, and what was shown to the patient.

Trace item	Recommended storage
Raw measurement	Measurement table + timestamps
Analysis period	health_analyses.period_start/end
Analysis engine	health_analyses.engine_version
Clinical rules	health_analyses.rule_version
LLM model	ai_health_feedback.model_name
Prompt version	ai_health_feedback.prompt_version
LLM context	ai_health_feedback.context
LLM response	ai_health_feedback.response
User-visible action	audit_logs where appropriate

21. Testing Strategy

Testing should be layered.

Layer	Examples
Database	Constraints, foreign keys, indexes, migrations
API	Authentication, authorization, validation, idempotency
Analytics	Averages, rolling windows, missing data, trend calculations

Layer	Examples
Clinical rules	Clinician-approved boundary and safety cases
LLM	Schema compliance, prohibited actions, hallucination resistance
Offline sync	Retries, duplicates, timestamp preservation, conflicts
End-to-end	Flutter → Node → analysis → LLM → Flutter
Security	Access control, secret handling, auditability

Create a fixed regression dataset of synthetic/de-identified cases. Every change to the analysis engine or rules should run against this dataset before deployment.

22. Recommended Implementation Roadmap

Phase	Deliverables
1. Health data foundation	BP, glucose, weight, conditions, medications, symptoms
2. Flutter entry/history	Offline entry, local storage, sync queue, graphs
3. Node APIs	Health CRUD APIs, authorization, idempotent sync
4. Python analytics	Aggregates, rolling averages, trend detection, feature output
5. Clinical rule engine	Versioned clinician-reviewed rules and safety flags
6. Analysis API	Persist health_analyses and expose summary endpoints
7. LLM integration	Context builder, structured output, validation
8. AI conversation	Relevant memory/context retrieval and chat UI
9. Safety/observability	Audit trails, fallback behavior, monitoring
10. Future ML	Only after validated use case, adequate data and governance

23. Suggested MVP Scope

For the first working version, avoid implementing every telemedicine feature at once. The health-analysis MVP can be:

```
Authentication
   ↓
Patient profile
   ↓
Blood pressure entry/history
   ↓
Glucose entry/history
   ↓
Weight entry/history
   ↓
Medication list
   ↓
30-day analysis
   ↓
Trend graphs
   ↓
AI explanation
   ↓
Clinician discussion prompts
```

Appointments, pharmacy, payments and other platform modules can continue in parallel, but they do not need to block validation of the health-analysis pipeline.

24. Recommended Final Data Relationship Overview

```
users
 ■■■ patients
 ■     ■■■ patient_conditions
 ■     ■■■ blood_pressure_readings
 ■     ■■■ glucose_readings
 ■     ■■■ weight_readings
 ■     ■■■ symptom_records
 ■     ■■■ patient_medications
 ■     ■       ■■■ medication_events
 ■     ■■■ health_devices
 ■     ■■■ medical_records
 ■     ■■■ lab_reports
 ■     ■■■ health_analyses
 ■     ■       ■■■ ai_health_feedback
 ■     ■■■ ai_conversations
 ■             ■■■ ai_messages
 ■
 ■■■ doctors
       ■■■ qualifications
       ■■■ documents
       ■■■ specializations
       ■■■ languages
       ■■■ facilities

patients
 ■■■ appointments
        ■■■ consultations
              ■■■ prescriptions
                    ■■■ prescription_items

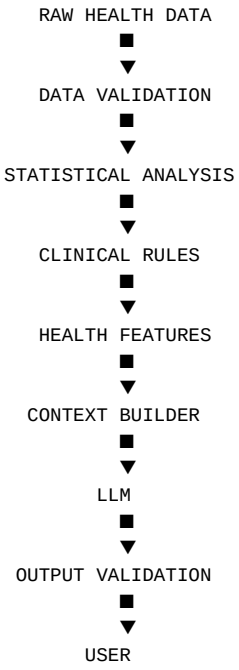
prescription_items
 ■■■ patient_medications (optional source link)
```

25. Architectural Decision Summary

Decision	Rationale
Separate BP/glucose/weight tables	Preserves context and supports independent time-series analysis
Keep prescriptions separate from active medications	Distinguishes historical prescribing from current treatment
Python for analytics	Strong statistical/time-series ecosystem and clear separation of concerns
Rules before LLM	Makes clinical logic deterministic, testable and auditable
LLM for explanation	Uses the model where language generation is strongest
Structured LLM output	Allows validation and safe Flutter rendering
Version analysis/rules/prompts	Supports reproducibility and auditing
Local offline queue	Fits rural/offline-first requirements
Idempotent synchronization	Prevents duplicate health records
Database as source of truth	Prevents conversational memory from becoming medical history
Safety validator	Prevents arbitrary model output from directly reaching the patient

26. Final System Principle

> The recommended SwastyaSathi architecture is not an LLM-centered health system. It is a health-data platform with deterministic analytics and an AI communication layer.



This separation gives the project a practical path from a student/prototype implementation toward a more robust telehealth platform: Flutter handles interaction and offline capture; Node.js handles the application API; PostgreSQL remains the source of truth; Python performs analytics; clinician-reviewed rules provide structured health interpretation and safety flags; and the LLM explains those results in an accessible way.

Appendix A — Minimal Change Set to the Current Schema

If development time is limited, prioritize these changes first:

- Replace or deprecate the generic vitals table.
- Add blood_pressure_readings.
- Add glucose_readings.
- Add weight_readings.
- Add patient_medications.
- Add patient_conditions.
- Add symptom_records.
- Add health_analyses.
- Add ai_health_feedback if AI feedback is included in the MVP.
- Implement measured_at + created_at and idempotent synchronization.
- Keep existing prescriptions as the historical prescribing record.

Appendix B — Example Analysis Result

```
{
  "analysis_version": "1.0",
  "period": {
    "start": "2026-09-01T00:00:00+05:45",
    "end": "2026-09-30T23:59:59+05:45"
  },
```

```
  "blood_pressure": {
    "measurement_count": 28,
    "latest": {
      "systolic": 138,
      "diastolic": 88
    },
    "average_7d": {
      "systolic": 134,
      "diastolic": 85
    },
    "average_30d": {
      "systolic": 134,
      "diastolic": 84
    },
    "trend": "increasing"
  },
  "glucose": {
    "fasting_measurement_count": 20,
    "fasting_average_30d": 128,
    "trend": "stable"
  },
  "weight": {
    "current_kg": 71.2,
    "change_30d_kg": -1.3
  },
  "flags": []
}
```

This example is illustrative only. Clinical interpretation and thresholds must be defined and reviewed using the appropriate current clinical guidance before production use.

End of Document